Chapter 5

Parsing Machine Learning and Deep Learning

IN THIS CHAPTER

- **Defining AI terms**
- **Processing natural language**
- **Interpreting transformers and tokens**
- **Knowing the limitations of AI**
- **Practicing responsible AI**

In this chapter, we cover some of the basics of machine learning, deep learning, and generative AI. Although it's fully possible to use AI tools such as ChatGPT without understanding the science and math behind them, a basic understanding of how machine learning and generative AI work will help you use these tools more effectively.

Decoding Machine and Deep Learning

When you first encounter a generative AI tool such as ChatGPT, it can seem like magic. Some people even speculate that generative AI tools are conscious and capable of thinking and having emotions. Knowing how these tools were created and how they work will quickly dispel that notion — or make you think differently about consciousness, but that's a subject for a philosophy book, not a coding book.

Defining key concepts

Before you can dip your toes into understanding how the latest AI systems are capable of generating complex responses to natural-language input, we need to present some vocabulary. People working with or writing about the systems we talk about in this book often use the terms *AI, machine learning, deep learning,* and *generative AI* interchangeably, but these fundamental terms are different (see Figure 5-1):

- **Artificial intelligence (AI)** is the use of computer science and data to solve problems. AI encompasses everything from expert systems and decision trees, which simulate the judgment and behavior of humans using a complex series of if-then statements, to machine learning, computer vision, and natural-language processing.
- **Machine learning** is a type of AI that focuses on developing and using computer systems that can learn and adapt without following explicit instructions. Machine learning can solve problems that would be prohibitively expensive to solve by programmers writing the algorithms by hand.
- **Deep learning** is a type of machine learning based on artificial neural networks. The word *deep* in *deep learning* doesn't indicate that it produces inherently more profound or mysterious AI. Rather, it refers to the use of multiple layers of algorithms (artificial neurons) in the neural network. We explain artificial neural networks and layers in the next section.
- **Generative AI (GenAI)** are AI systems that can generate new content based on the data used to train them. Some form of GenAI has been around since the 1960s (for example, ELIZA a text-based chatbot created by Joseph Weizenbaum, which you find in the section "History of NLP" later in the chapter). In recent years, the content created by GenAI is of a high enough quality to be more than a novelty, thanks to the use of deep learning.

Thinking about neural networks

The idea behind deep learning is to teach computers to process data based on how we think our brains work. In a human brain, cells called *neurons* form a complex and massive interconnected network. Using chemical reactions and electrical currents, neurons send signals to each other to enable us to learn and process information.

Neural networks in deep learning, also known as *simulated neural networks (SNNs)* or *artificial neural networks (ANNs),* consist of artificial

neurons called nodes that form layers, as shown in Figure 5-2.

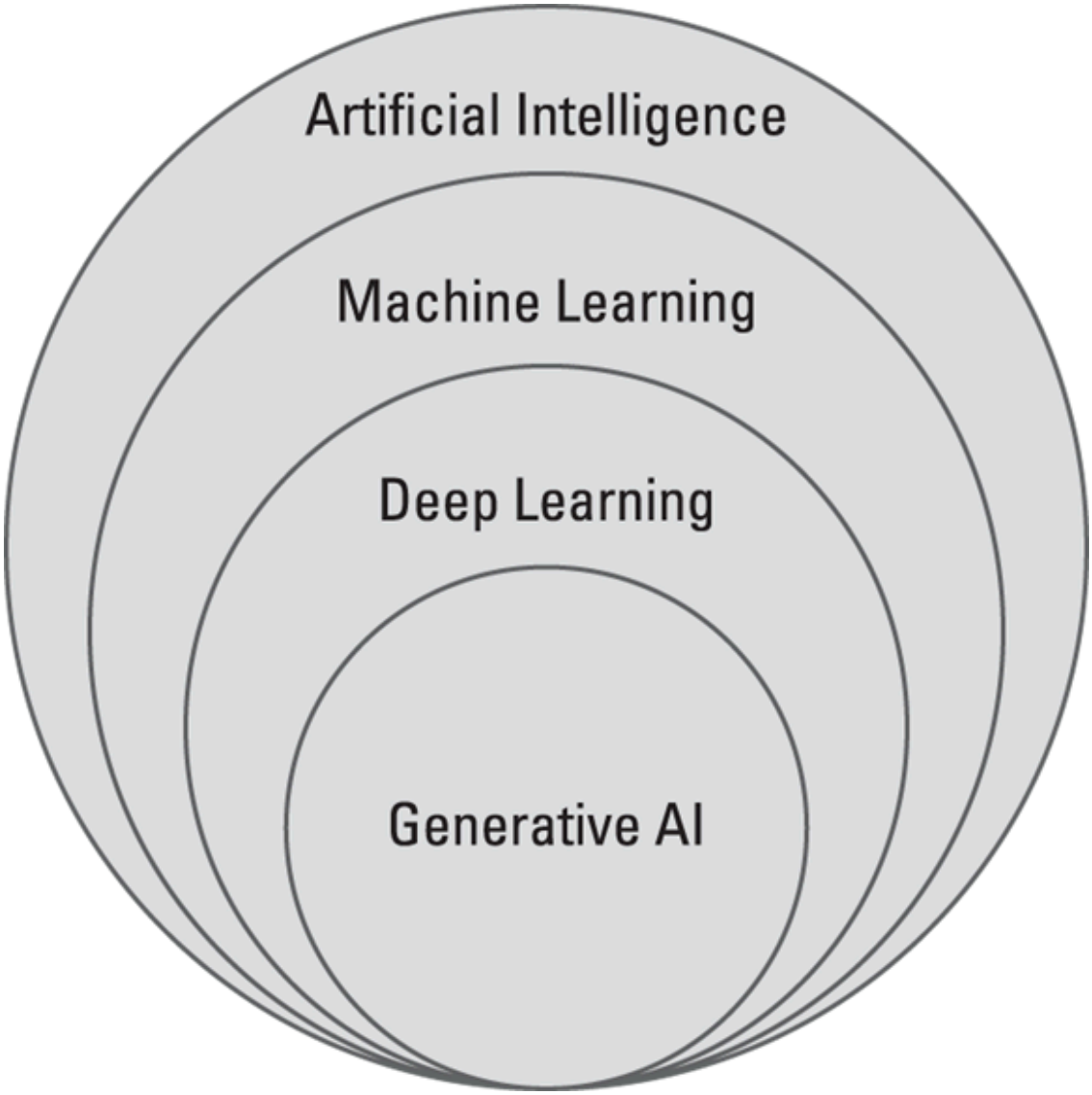


FIGURE 5-1: The relationship between fields in AI.

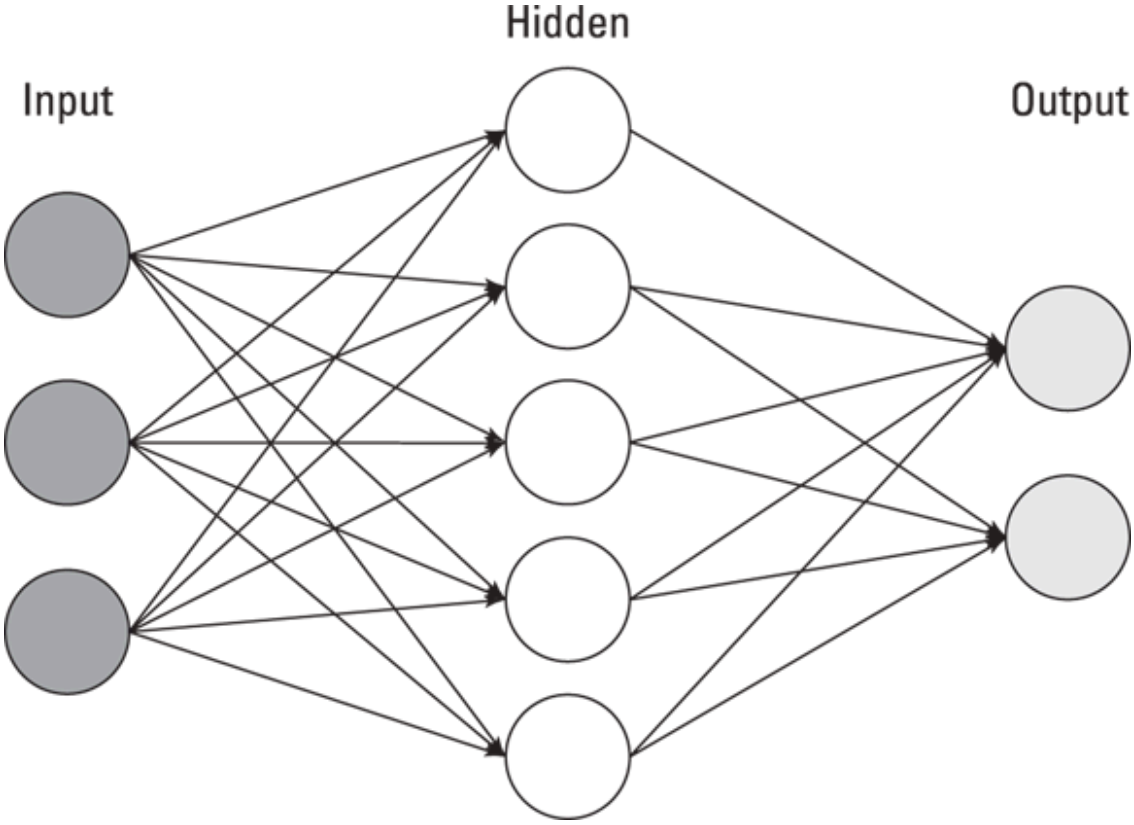


FIGURE 5-2: Nodes are arranged in layers.

Each *node* is a software module that processes some input and determines whether to pass it on to the next layer of neurons for further processing. A simple neural network consists of three layers: the input layer, the hidden layer, and the output layer.

Input layer

The *input layer* receives data and puts it in a format that the hidden layers can use. For example, in a neural network for analyzing images, the images are first converted to the same size and dimensions. Next, the input layer takes in the pixel values (the amount of red, green, and blue) for each image and passes them along to the hidden layer.



In the preceding example, the number of neurons in the input layer is determined by the number of pixels in the image. If the neural network will be processing color images, the number of pixels is multiplied by 3 for each of the pixel values (red, green, and blue) to get the number of neurons. For example, the image shown in Figure 5-3 has a

width of 56 pixels and a height of 56 pixels. We've magnified the image so you can see the individual pixels. An input layer for working with this image would have 56 × 56 × 3, or 9,408, neurons.



Czar / Wikimedia Commons / Public Domain

FIGURE 5-3: A color image containing 3,136 pixels requires a 9,408-neuron input layer.

Hidden layer

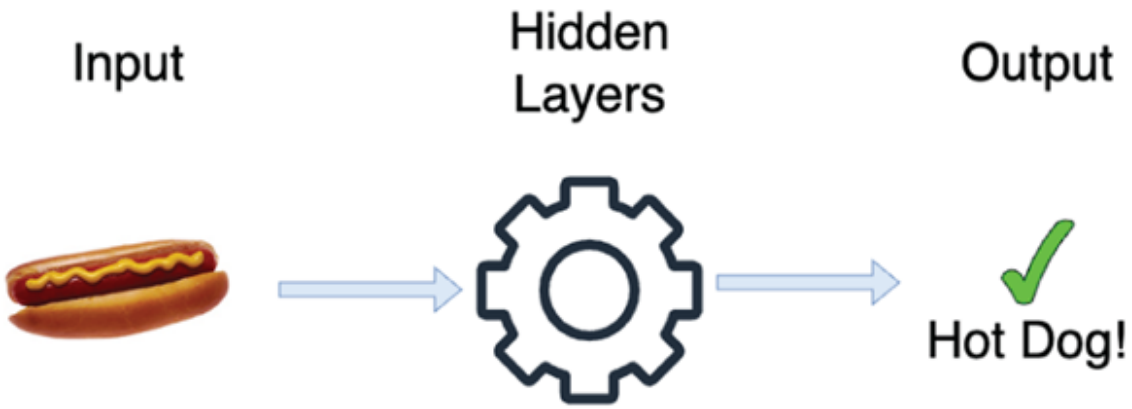
A *hidden layer* receives data from the input layer or other hidden layers and processes it further to extract features from the image, such as color, shape, and texture. More complex tasks require more neurons in the hidden layers. A neural network can have many hidden layers.

Output layer

Neural networks must have at least one *output layer*, which provides the final result of the calculations from the hidden layers.

Figure 5-4 shows a simple artificial neural network that determines whether or not a photo contains a hot dog. This type of problem is known as a binary classification because the output from this neural network is either 1 (hot dog) or 0 (not hot dog).

But how does the neural network determine whether the photo contains a hot dog? That's where its training comes in.



Czar / Wikimedia Commons / Public Domain

FIGURE 5-4: Is it a hot dog?

Training and testing models

Before a neural network can perform a task, it must be trained. In the case of the hot dog classifier, the neural network can be trained by analyzing thousands of pictures of hot dogs as well as an equal number of things that aren't hot dogs. Each photo must be labeled ("hot dog" or "not hot dog"). From these pictures, the neural network can make certain conclusions about what a picture of a hot dog contains. For example, a picture of a hot dog generally will contain a lighter-colored area (the bun) surrounding a darker-colored cylindrical shape (the hot dog), as shown in Figure 5-5.



FIGURE 5-5: Many pictures of hot dogs have similar characteristics.

As the layers of the neural network analyze pictures, values called weights and biases are assigned to each neuron in the network. The *weight* determines the strength of the connection between two nodes in the network. The *bias* determines the threshold at which a node is activated. Weights and biases adjust how much a neuron will contribute to the final result.

The weights and biases that the model learns from the training data are called *parameters*. The complexity of a model can be described by how many parameters it has. More complex models have more parameters because they're able to learn more complex patterns in data. You can think of parameters as similar to the synapses that connect neurons in a human brain.

In machine learning, a *model* is a mathematical representation of a real-world system or phenomenon. Some examples of models follow:

- **Scientific models** make predictions involving the atmosphere, diseases, and the universe.
- **Engineering models** design and test new products.
- **Demand models** predict how many units of a product a business will sell.
- **Financial models** predict the performance of stocks and other financial instruments.

How accurately a model can make predictions (whether or not a new picture contains a hot dog, for example) is based on the quality and quantity of data that went into its training.

Small models that make relatively simple predictions, such as whether or not a photograph contains a hot dog, can be created quickly, and the risks of messing up such a model are unlikely to be catastrophic.



TIP The most time-consuming part of creating a model is usually the gathering and labeling of the datasets. Many publicly available free datasets exist. You can find a list of them at `https://openml.org`.

Complex models, such as climate models and models of systems of the human body, can take months or years to train, and getting it wrong can have expensive or life-threatening consequences. Some of the challenges of training neural networks include the following:

- **Complex neural networks can involve terabytes of training data and billions of parameters.** Training a large neural network is computationally expensive.
- **Neural networks can learn their training data too well, which is called *overfitting*.** When overfitting occurs, the model will be able to ace tests about the data it was trained on but will have a hard time generalizing to new data (which is the point of a model).
- **Interpretability (understanding how a model makes predictions) can be difficult.** Because of the number of parameters in a model, people can have difficulty figuring out how a neural network makes its predictions.

Demystifying Natural-Language Processing

Natural-language processing (NLP) is the branch of AI concerned with giving computers the capability to understand human language in written and spoken form. NLP can be further divided into two subsets:

- **Natural-language understanding (NLU)** involves the analyzing of text and speech to figure out the meaning of sentences.
- **Natural-language generation (NLG)** is concerned with giving computers the capability to write.

NLP is used for a wide variety of purposes, including the following:

- **Machine translation:** Translating text from one language to another
- **Speech recognition:** Understanding spoken language and translating it into text
- **Text summarization:** Generating summaries of longer texts, such as books, articles, or briefs
- **Question answering:** Answering questions asked in natural language
- **Sentiment analysis:** Identifying the sentiment of text, for example, whether it's positive, negative, or neutral
- **Computer programming:** Writing or assisting with the writing of computer programs



REMEMBER Human language includes not only the languages we use for communicating with each other (such as English, French, and Greek) but also the languages we use for communicating with computers (such as C++, JavaScript, and Haskell).

History of NLP

Natural-language processing has been around since the first experiments in automatic translation in the 1950s. The creators of the first automatic translation machine predicted that the problem of machine translation would be solved within a few years. However, it ended up taking much longer, and we still haven't perfected it.

Early natural-language processing systems relied on a list of handwritten rules that the computer applied to the data it received. The ELIZA chatbot, created in the 1960s, could have a somewhat convincing conversation with a human by following rules and responding as a non-directional psychotherapist. Although ELIZA was created as a joke, it successfully demonstrated that natural-language conversations between people and computers were possible. Figure 5-6 shows a typical conversation with ELIZA.

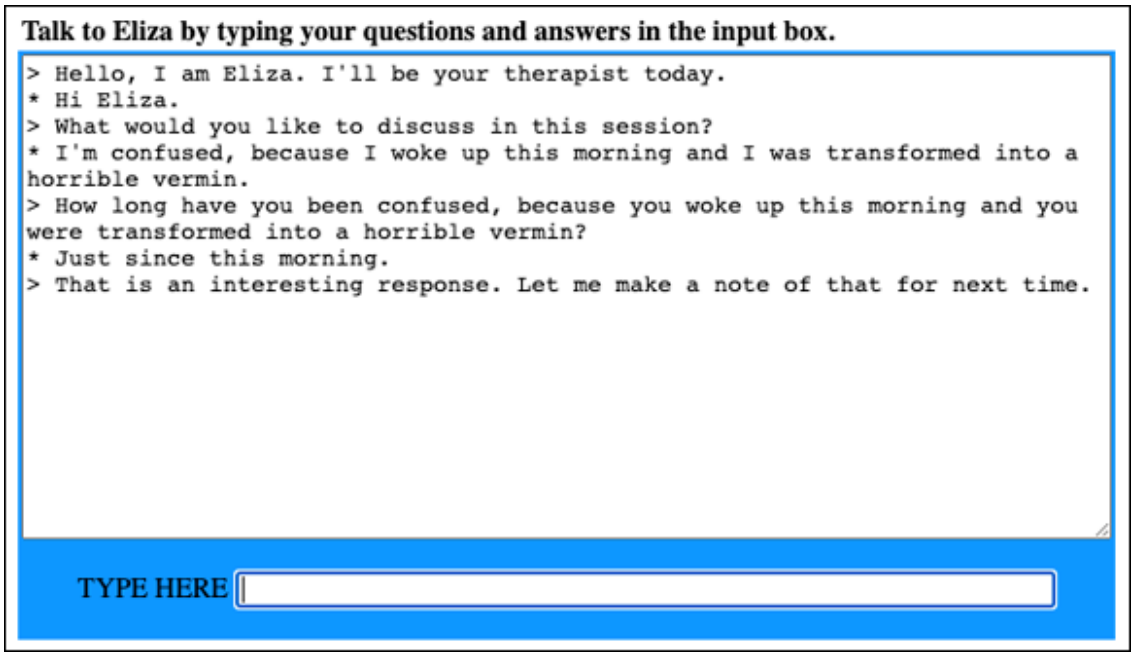


FIGURE 5-6: Early NLP was based on rules.

Machine learning algorithms and increased computing power led to advances and new techniques for natural-language processing starting in the late 1980s.

In the 1990s, Richard Wallace created the ALICE chatbot. ALICE is similar to ELIZA in that it uses pattern-matching rules. The operator of an ALICE chatbot could add rules, which made it useful for customer service applications such as answering questions about a company's products. Figure 5-7 shows an example conversation with ALICE.

The growth of the internet made large collections of text readily available for training models. As a result of all this data, new techniques were created in the 1990s and 2000s that depended on statistical analysis of large datasets.



FIGURE 5-7: An example of an Alice chatbot.

Machine learning models made it possible for natural-language processing systems to be trained without having to hand-code rules. Further advances in machine learning, including neural networks, made virtual assistants such as Apple's Siri and Amazon's Alexa possible in the 2010s and eventually enabled the remarkable advances in natural-language processing that made tools such as ChatGPT possible.

Overcoming the challenges of NLP

In the past, creators of NLP systems focused on breaking down human languages into rules. The idea was that if you could give a computer access to a language's dictionary and a grammar book, it would know that language.

In the case of programming languages, this approach was somewhat successful. If you know the rules of a programming language and the right keywords to use, you can at least write statements.

But human language is more complex. Here are just a few of the things that a computer can't understand by memorizing a dictionary and grammar:

- Dialects
- Accents

- Sarcasm
- Metaphors
- Humor
- Grammar and usage exceptions
- Homophones

The fact that a box of chips and wires can understand any of what people say is incredible. Until recently, it seemed that computers were doomed to understand only a formal and limited subset of human languages and that chatting with an AI assistant would always be a frustrating and disappointing experience.

So what changed? The biggest breakthrough in NLP was when researchers decided to throw out the rules and start teaching computers to talk the way humans teach babies to talk, namely, by exposing them to language and letting them figure things out.

Understanding supervised and unsupervised learning

Supervised learning relies on *labeled data,* which is data annotated with tags that describe what the data is. For example, if you want to train a model to recognize spam email messages, you could create a dataset containing millions of email messages, each labeled either "spam" or "not spam." However, the process of labeling data can be costly and time consuming.

Unsupervised learning is the finding of patterns in unlabeled data. With unsupervised learning, algorithms sort through unlabeled data looking for patterns. Unsupervised language-learning models can be trained on very large datasets to create large language models (LLMs). Unlike supervised learning, unsupervised learning can be done inexpensively and quickly.

Language generation techniques

Natural-language processing techniques can be divided into traditional machine learning methods and deep learning methods. Traditional machine learning techniques include the following:

- **Logistic regression** is a classification algorithm that aims to predict the probability that an event will occur based on some input.
- **Naïve Bayes classifiers** are a collection of statistical classification algorithms based on Bayes's theorem, which describes the probability of an event based on prior knowledge of conditions. The *naïve* part of the name refers to the assumption these algorithms make that individual words are not dependent on each other.
- **Decision trees** work by splitting a dataset based on different inputs. For example, if you wanted to find out whether it's likely to rain, you might start by asking whether it's cloudy. If so, you might then ask about the humidity and then the temperature. After several splits, you can come up with a prediction.
- **Latent Dirichlet allocation (LDA)** is used for topic modeling. Topic modeling techniques scan a set of documents to detect patterns and cluster together word groups that best characterize the set.
- **Hidden Markov models (HMMs)** decide the next state of a system based on the previously observed state. The *hidden* part of the name refers to data properties that aren't directly observed. In natural-language processing, the hidden state is the parts of speech, and the observed state is the words in a sentence.



Some of the techniques used in machine learning are named after the people who formulated the theorems the techniques rely on. Thomas Bayes was an eighteenth-century English statistician, philosopher, and minister. Peter Gustav Lejeune Dirichlet was a nineteenth-century German mathematician. Andrey Markov was a Russian mathematician who lived at the end of the nineteenth century and the beginning of the twentieth century.

While traditional NLP methods are often able to do a good job, it wasn't until deep learning techniques were applied to NLP that things got interesting. Following are some deep learning NLP techniques:

- **Convolutional neural networks (CNNs)** were designed to be used for working with images but can also be used with documents. In NLP, CNNs treat documents as images made up of sentences instead of pixels.
- **Recurrent neural networks (RNNs)** use hidden states to remember previous information. Because of this, they can learn how every word in a sentence is dependent on previous words or words in the previous sentence.

- **Transformer models** learn context by tracking relationships in sequential data (such as the words in a sentence). Transformer models are the breakthrough that has revolutionized NLP in recent years. I tell you more about how transformers work in the next section.

Understanding Transformers

Transformer models use a self-attention mechanism to find dependencies between inputs and outputs. To understand what that means, you first need to know what attention and self-attention are in machine learning. Read on!

Learning to pay attention

The mathematical technique that transformer models use is called attention. The goal of *attention* is to allow the model to focus on important parts of the input while generating its output. As people, we do this naturally. When you read a sentence or look at an image, you can easily see which parts of the sentence or image are the most important in terms of understanding it.

When the idea of attention first became popular, it was combined with recurrent neural networks (RNNs). But RNN models have to consider words in sequence one at a time, which is a slow process. Even worse, RNNs tend to assign more importance to recent words and the ends of sentences.

Another side effect of considering words one at a time is that you lose important context. Consider this sentence:

> *The player swung his bat, and he ran to first base.*

Anyone with a passing familiarity with baseball will know that *bat* refers to a baseball bat and that *he* refers to the player. Considered one word at a time, however, a language model can't make the same connections.

This is where the idea of self-attention comes in. *Self-attention* allows a model to learn information about an input sequence from the input se-

quence itself. With self-attention, a transformer model finds relationships between the words *player* and *bat* as well as between *player* and *he*.

Figure 5-8 shows a visualization of the connections a transformer model makes between the word *he* in the example sentence and the other words in the sentence. Darker lines indicate a stronger connection.

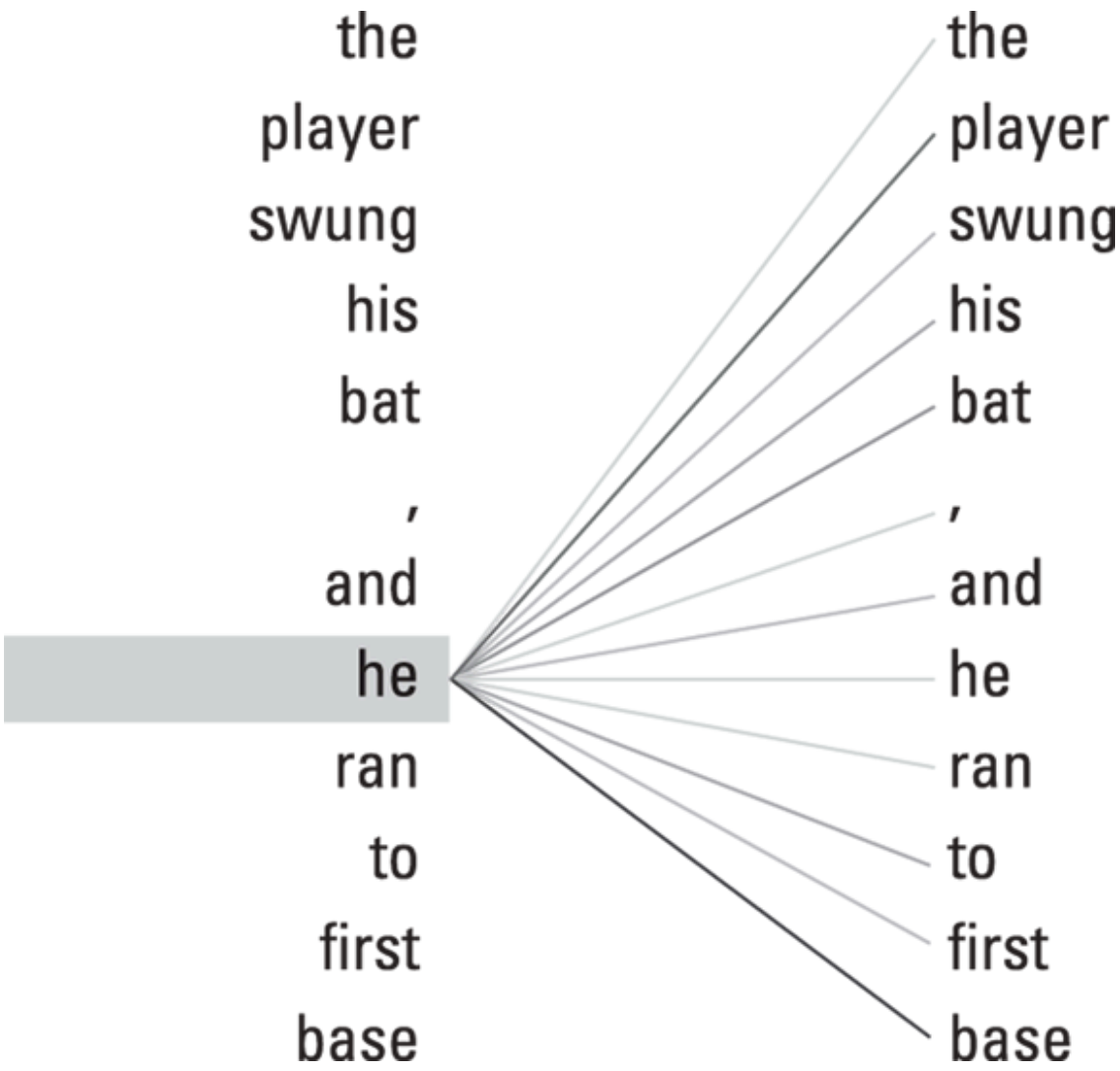


FIGURE 5-8: Visualizing self-attention.



The visualization in Figure 5-8 was created using a tool called BertViz. You can learn more about how BertViz works and try it out yourself at `https://github.com/jessevig/bertviz`.

Self-attention was first proposed as a solution for improving large language models in the paper "Attention Is All You Need." This paper, pub-

lished in 2017, led to the giant leaps forward in transformer models that we've seen in recent years.

Getting tokens

You may be surprised to learn that language models can understand only numbers; they don't read or understand words as we do. When you ask an NLP system a question, your input must first be converted into a sequence of numbers called *tokens*. You can think of tokens as the language equivalent of pixels in an image.

These tokens are read by a model's input layer and then processed through the hidden layers to predict and output responses to your prompts.



REMEMBER Hidden layers are the layers between the input and output layers that process data and learn features of it.

Text can be converted to tokens in a variety of ways. One common method is *word tokenization,* which simply creates a token for each word in the text. Transformer models use *sub-word tokenization,* which converts text into common sequences of characters, such as *token* and *ize.* Using smaller units improves the model's performance.

To see how OpenAI converts text to tokens, go to OpenAI's Tokenizer at `https://platform.openai.com/tokenizer`. Figure 5-9 shows the list of tokens that were generated when I entered the sentence *Transformer models use sub-word tokenization.*

After the text is tokenized, it's encoded into token IDs. The token IDs created from the text in Figure 5-9 are shown in Figure 5-10.



Token IDs are input into a neural network as a sequence of numbers called a *numerical vector*.

Being aware of tokens and the tokenization of your input is important. If you're using a model through OpenAI's API, the combined number of tokens in your input and the model's response determines how much you pay for the service.



ChatGPT

FIGURE 5-9: Tokenizing a prompt.



ChatGPT

FIGURE 5-10: Tokens are represented as token IDs.

In addition, the NLP system you're using may have a *token limit,* which is the maximum number of tokens the model will consider while generating its response. Your input and the model's responses contribute to the number of tokens used by a conversation. In a long conversation with ChatGPT, or if you input an excessively long prompt, the token limit may be exceeded. In this case, the model will "forget" earlier prompts in the conversation, and you'll need to repeat any instructions you included at the beginning.

Token limits are imposed on a model to prevent it from running out of memory, to shorten the model's response time, and to reduce the amount of computation required to train and deploy the model.

Table 5-1 lists the token limits of standard versions of several models. Many models also have extended versions with larger token limits.

TABLE 5-1 Token Limits

Model	*Creator*	*Token limit*
GPT-2	OpenAI	1024
GPT-3	OpenAI	2048
GPT-4	OpenAI	8000

Model	*Creator*	*Token limit*
BERT (base)	Google	512
RoBERTa (base)	Facebook	512
T5 (base)	Google	512
XLNet (base)	Google/CMU	512
Electra (base)	Google	512
DistilBERT	Hugging Face	512

Illuminating Generative AI Models

A *generative AI model* is a model that is trained on content (such as images or text) and can use that content to make predictions to generate new content. OpenAI's GPT-3, the model behind the first public version of ChatGPT, is a generative model for natural-language processing. By leveraging what it's learned about how people talk and write, it can accurately predict what the next word or sentence should be in response to input.

For example, if you provide the words *peanut butter and* to GPT-3, it will most likely predict that the next words should be *jelly sandwich.* GPT-3 doesn't have any experience with eating or making peanut butter and jelly sandwiches, but it has analyzed a lot of text, and the most common way to finish the phrase *peanut butter and* is with *jelly sandwich.*

Like the hot dog photo classifier (which I introduce in the section "Output layer" earlier in the chapter), a generative AI model is only as good as its training data. GPT-3 was trained with over 45 terabytes of text data, so it has amazing capabilities, such as taking context into consideration when responding to input.

Generative AI models have been steadily getting better as a result of being trained on more and higher-quality data. Table 5-2 compares the size of some recent generative models.

TABLE 5-2 Parameters in Generative AI Models

Model	*Developer*	*Parameters*
Gato	DeepMind	1.18 billion
ESMFold	Meta AI	15 billion
LaMDA	Google	137 billion
GPT-3	OpenAI	175 billion
Bloom	Hugging Face and BigScience	176 billion
MT-NLG	Nvidia and Microsoft	530 billion
WuDao 2.0	Beijing Academy of Artificial Intelligence	1.75 trillion
GPT-4	OpenAI	1.76 trillion



REMEMBER As I mention earlier in the chapter, parameters are the values that the model has learned from its training.

Recognizing AI's Limitations

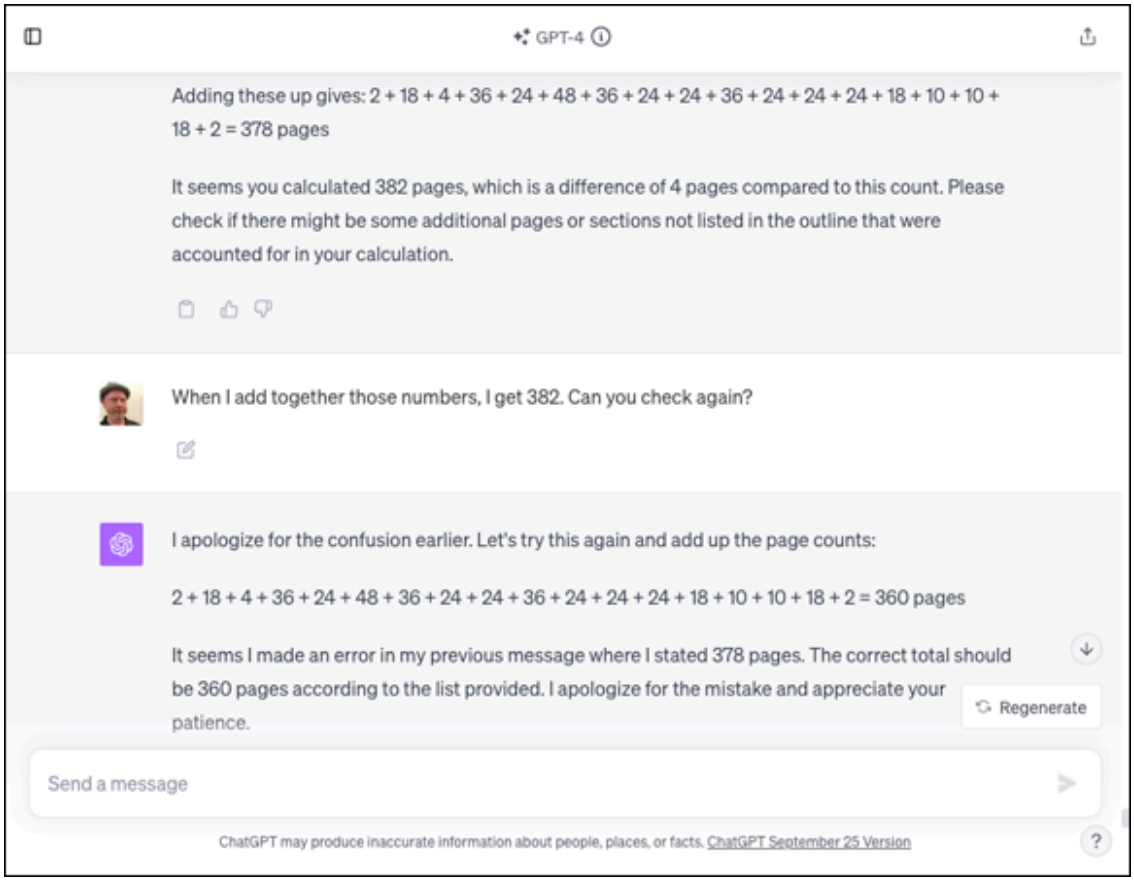
As impressive as they are, transformer models don't know how to talk — they know only how to look for patterns in sequential data (such as sentences, statements, or functions). When you train a generative model on enough data, it becomes very good at finding patterns and making predic-

tions, but it does have limitations, and you should never trust the output of a chatbot (or any AI system) entirely.

Language models are bad at math

AI chatbots are language models tuned for conversation. If you ask a language model for the answer to a basic math problem, it will usually respond confidently with some answer. However, upon checking that answer using a calculator, you may be surprised that it's often just plain wrong. Currently, generative models are able to make predictions based only on content they've previously seen. If you give them a math problem they've never seen before (no matter how trivial), they'll respond based on the answers to similar math questions in their training data rather than by doing the math the way a calculator would.

For example, Figure 5-11 shows a conversation I had with ChatGPT when I was planning the outline for this book.



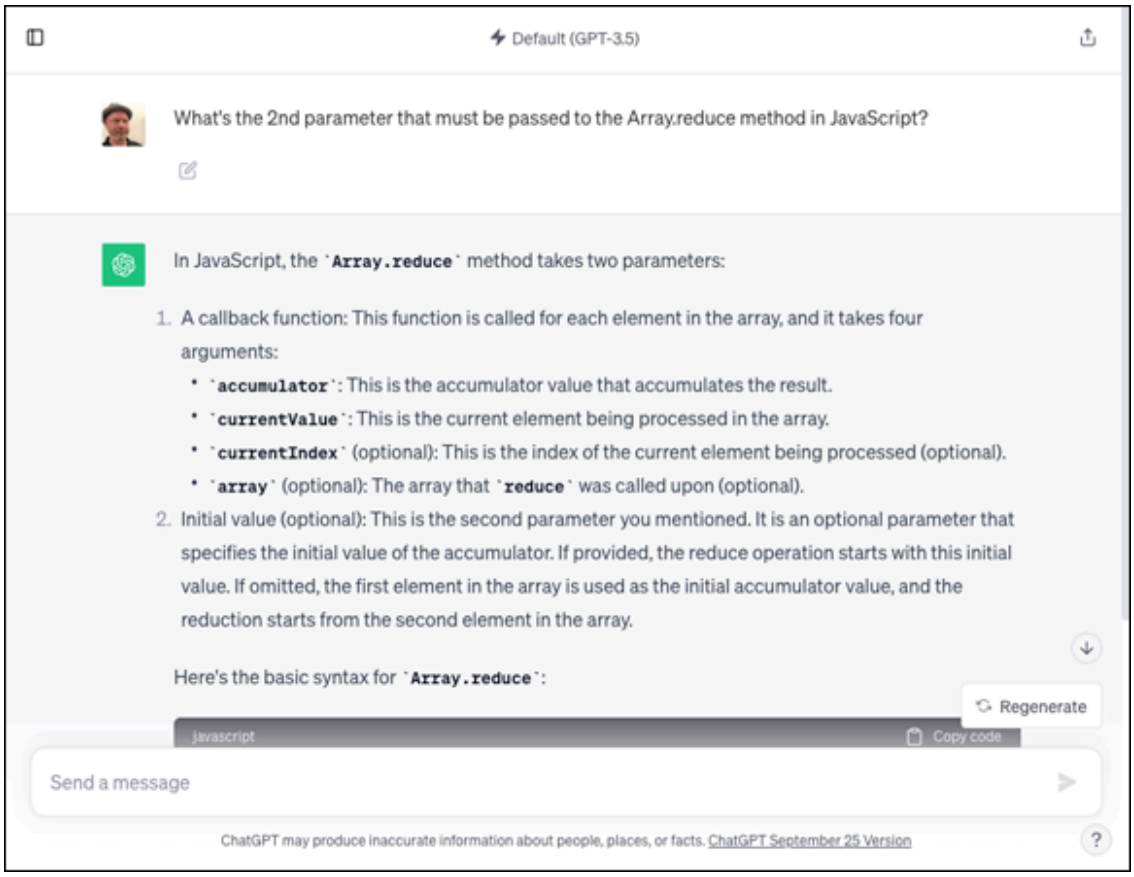
ChatGPT

FIGURE 5-11: GPT-4 gets an addition problem wrong.

Language models are wordy

AI chatbots often respond to prompts with paragraphs when a one-word answer will do. The standard ChatGPT response to even a simple question reads like a high school book report, containing an introduction, an analysis of an issue from multiple viewpoints, and a summary.

Figure 5-12 shows ChatGPT's nearly 200-word response to a question that any JavaScript programmer could answer with two words.



ChatGPT

FIGURE 5-12: ChatGPT uses 200 words where 2 will do.

AI has limited knowledge

Although it's now possible for ChatGPT to access data on the internet, the model behind ChatGPT is finite. As a result, ChatGPT doesn't know everything. When prompted with questions about obscure or recent topics, the answer the model returns may be wrong.

AI lacks common sense

No machine learning model has had the experience of being human. As a result, the responses generated by the model lack common sense. For example, if you ask ChatGPT how to swim to the moon, it may provide an answer without questioning the absurdity of the question. Any human would first question the value of such a strange question.

AI has accuracy issues

The accuracy of responses generated by a model depends on many factors, including the training data, context, user input, complexity of the prompt and language, and bias. As the user of a model, you have control over only some of these factors. Where possible, however, you can help the model provide better responses by knowing how best to prompt it and by challenging the model's output in follow-up prompts.

AI has the potential to be biased

Because they're trained largely on text written by people, machine learning models will pick up the biases and preferences that exist in the training data. Creators of models put a lot of effort into eliminating bias — which is a worthy goal but an impossible task.



WARNING Unintended consequences and even dangerous situations may result from bias in models. The classic example is when Microsoft released its Tay chatbot to the internet in 2016. Within one day of talking to people, the chatbot went from saying things like “Humans are cool” to making racist and sexist comments.